Lattice: Signed, Replayable Capability Receipts for Model Runs

Lakshman Turlapati*
Full Self Browsing
https://github.com/fullselfbrowsing/Lattice
Alumnus, Naveen Jindal School of Management,
The University of Texas at Dallas (M.S., December 2025)
lakshmanturlapati@gmail.com

Rabih Neouchi
Naveen Jindal School of Management
The University of Texas at Dallas, Richardson, Texas, USA
Rabih.Neouchi@UTDallas.edu

***Abstract*: Production systems built on large language models usually keep a free-text log of a run. Lattice is a capability-first TypeScript runtime. When a signer is configured, `ai.run` and multi-agent crews emit capability receipts. A receipt commits to hashes of the input artifacts and the output, a hash of the contract, and the selected route. It does not commit to the task text, the policy, or the reasons a route was chosen. The body is canonicalized with RFC 8785, matched personal data is masked before signing, and the envelope is signed with Ed25519. Routing is a deterministic function, and a violated output invariant is terminal. An offline evaluation against mocked providers measures signing cost, receipt size, tamper rejection, and whether a kept output still matches its signed hash. Audience and nonce binding, a schema-version floor, and a chain-length check are verifier opt-ins. A crew's closing receipt lists every other member of its chain, so a verifier that enforces closure rejects a dropped leaf.**

***Index Terms*: capability receipts, verifiable replay, LLM orchestration, software supply chain, canonical JSON, digital signatures, reproducibility, audit trails**

I. Introduction

Software that calls large language models has moved from prototypes into systems that approve refunds, triage insurance claims, route field operations, and process medical and recruiting documents. These systems combine text with images, audio, documents, structured outputs, and tool calls, and they increasingly spread a single task across several model providers for cost, latency, privacy, or capability reasons. As the stakes rise, two questions follow every run. First, can an operator or an auditor trust the record of what happened. Second, can anyone later check, offline and without trusting whoever stored the record, that a kept input or output is the one the run used or returned. Today the honest answer to both questions is usually no.

The reason is structural. The dominant integration libraries are provider wrappers. They normalize the shape of a chat or generation call across vendors and they provide convenience around tools, memory, and agents. They do not produce a tamper-evident record of a run, they do not return a deterministic and inspectable account of why one model was chosen over another, and they do not let a third party check a kept result against a signed artifact. A free text log is not a proof. It can be edited after the fact, it does not bind the inputs to the outputs, and it carries no signature that a verifier can check with a public key.

This paper argues that the missing primitive is a per-run, signed, replayable *capability receipt*, and that a runtime can make such a receipt a default byproduct of normal operation rather than an after-the-fact instrumentation task. We build this argument concretely through Lattice, a capability-first runtime SDK written in TypeScript. A developer describes the job, supplies any mix of artifacts, declares the desired outputs and policy constraints, and optionally attaches a contract. Lattice then routes the work deterministically, packs context, prepares artifacts for the chosen provider, and enforces output invariants. When a signer is configured, `ai.run` emits a receipt on every outcome, success or failure, and a multi-agent crew emits a chain of receipts. The single-agent loop, `ai.runAgent`, returns no receipt.

Lattice provides four guarantees that together make a run auditable and reproducible.

1) **Deterministic and inspectable routing.** The router is a pure function of the capability catalog, the request, and the policy. It uses hard filters, a stable score, and a deterministic tie break, and its execution plan reports the full set of candidates, the rejected candidates, the fallback chain, and a typed list of reasons when no route is possible. There is no randomness and no model call inside routing. The receipt signs only the selected route.
2) **Terminal output invariants.** Output level tripwires such as "must cite an artifact" or "no matched email, social security, phone, or card number" are evaluated by a pure kernel. A violation produces a terminal error that the fallback chain is forbidden to retry, which prevents a system from quietly trying again until a banned output slips through.
3) **Signed capability receipts.** With a signer, every `ai.run` outcome and every crew run emits receipts that are canonicalized with RFC 8785, redacted before signing, and signed with Ed25519 inside a Dead Simple Signing Envelope. A receipt binds hashes of the input artifacts and the output, the contract hash, and the

*Corresponding author: lakshmanturlapati@gmail.com.

selected route to a key, through a signature that any holder of the public key can check.

4) **Offline replay as output binding.** A receipt and its content-addressed artifacts can be materialized into a replay envelope. Materialization verifies the signature before it loads any artifact. Replay does not call a model: it recomputes the hash of the recorded output and compares it with the value the receipt commits to, and a property test checks that round trip.

We make the following contributions. We define the capability receipt as a signing and replay primitive for model runs, and we show how a redact-then-sign ordering and a canonical encoding turn it into a verifiable artifact rather than a log line. We show how a deterministic and inspectable router, terminal output invariants, and a content-addressed verification tool compose into a runtime where auditability and reproducibility are default properties. We describe a working implementation that ships publicly with supply-chain provenance, modular adoption paths, a seven-provider parity contract, and a test suite that encodes the guarantees as invariants. We also show how external executors can mint compatible receipts without adopting Lattice's provider or agent runtime. The remainder of the paper covers background and related work, the threat model and goals, the design, the implementation, the evaluation, a discussion, the limitations, future work, and a conclusion.

II. Background and Related Work

A. Provider wrappers and agent frameworks

A first family of systems normalizes model access. The Vercel AI SDK [1] offers a broad provider-agnostic TypeScript toolkit with multimodal generation, tools, and agents. LangChain and LangGraph [2] provide orchestration, sessions, and tracing. LiteLLM [3] is strong routing infrastructure that exposes a common interface across many providers with fallback and gateway features. The OpenAI Agents SDK [4] provides agent loops, sessions, and tool-use. The Model Context Protocol [5] standardizes how tools and context attach to a model session. These systems are valuable and widely used. None of them, however, emit a signed and replayable per run receipt. Their tracing is observability for humans, not a cryptographic artifact that a third party can verify against a public key, and their routing is generally either configuration driven or model chosen rather than a deterministic pure function whose full decision is returned for inspection.

B. Software supply-chain provenance

A second family secures the provenance of software artifacts. in-toto [6] provides end-to-end guarantees over a software supply chain by signing the steps that produced an artifact. The Dead Simple Signing Envelope [7] defines a minimal envelope and a pre-authentication encoding so that signatures commit unambiguously to a payload and its type. Sigstore [8] makes signing and transparency broadly accessible. SLSA [9] defines levels of supply chain integrity, and The Update Framework [10] addresses survivable key compromise in update systems. Lattice borrows directly from this lineage. It reuses the DSSE envelope and pre-authentication encoding, it adopts a key set with explicit key states in the spirit of key rotation and revocation, and it treats the published package itself as a supply chain artifact with provenance attestations. The novelty is to apply this provenance discipline not to a build, but to an individual model run at runtime.

C. Canonicalization and signatures

Signing a structured object requires a deterministic byte encoding, otherwise two semantically equal objects can produce different signatures. The JSON Canonicalization Scheme [11] defines such an encoding for JSON, and the I-JSON message format [12] constrains JSON to an interoperable subset, which matters for how numbers and floats are serialized. Lattice canonicalizes receipts with RFC 8785 and serializes monetary cost as an I-JSON string so that receipts never carry raw floats. Signatures use EdDSA [13], the elliptic curve signature construction that is widely deployed for compact and fast signatures.

D. Transparency and reproducibility in machine learning

A third family documents models and data for accountability. Model Cards [14] and Datasheets for Datasets [15] standardize disclosure about a model or dataset, and reproducibility programs in the research community [16] push toward runs that others can repeat. These efforts describe an artifact or a study. A capability receipt is complementary and finer grained. It describes a single production run and makes that run independently checkable and replayable. Merkle trees [17] provide a standard way to commit to a set of items with a single root; a Lattice receipt uses one for its signed lineage root over the artifacts that a run consumed and produced.

E. Signed receipts for agent actions

Four systems from 2026 target evidence for autonomous agent runs and are the closest prior work. Sello [18] has the service that receives an agent's call sign a receipt with its own Ed25519 key inside a COSE envelope, encrypt it to the agent's owner, and publish it to a witnessed Merkle log; verification needs the log inclusion proof and a key registry, inputs and outputs appear only as hashes, and the protocol is independent of models and providers. Agent Flight Recorder [19] is middleware that hash-chains agent events into Merkle epochs and anchors epoch roots on a public chain; there is no per-event signature, a verifier needs read access to the chain, and policy verdicts are recorded rather than enforced. NovaFabric [20] captures traffic to several providers into an operator-signed evidence bundle, with Ed25519 at the collector and an ECDSA P-256 seal per tenant inside a DSSE envelope, that is designed for offline verification with stock tools and supports four replay modes; its policy gates act on promotion, export, and replay rather than on live runs, and its author describes the implementation as experimental. Janus [21] is the closest in spirit: a single-node daemon appends each agent

step's proposal, gate verdict, and validator response to a hash-chained log whose segments it signs with Ed25519 before the step runs or its effect is released, an auditor re-derives every verdict offline from the log and one public key, and replay re-derived all 287 terminal sagas in the paper's experiment; it does not choose models or providers, model decision records are self-reported rather than signed, and effects on its SDK path are admitted cooperatively rather than held. TrustTrack [22] is a vision paper that proposes verifiable identity, policy commitments, and tamper-resistant behavioral logs for multi-agent systems, without an implementation. Certificate Transparency [23], [24] is the reference design for the log-based approach that Sello and Agent Flight Recorder build on: it makes issuance auditable after the fact through an append-only log, whereas a Lattice receipt is checkable on its own with a public key and binds the selected route, the contract hash, and the output hash. Table I compares the four implemented systems with Lattice, and Section VI-C runs each of them on the same recorded requests.

F. The gap

Across these families, and to our knowledge, Lattice is the first open SDK that combines per-run signed receipts, deterministic routing, run-time policy enforcement, and signed output binding in one runtime. The provider wrappers and agent frameworks route and orchestrate but do not sign; the supply chain systems sign and verify offline but know nothing about model runs; and the agent receipt systems in Table I sign and verify evidence, with Janus also gating effects and replaying decisions, but none of them chooses the model, so their evidence is never bound to a selected route. The receipt design first appears in the repository's public history in commit `fa6d5a1` of May 11, 2026 [25], before the Sello (June 2026) and the Agent Flight Recorder, NovaFabric, and Janus (September 2026) preprints. The audience, nonce, and expiry fields, chain verification, and chain closure were added in October 2026, after those preprints.

III. Threat Model and Goals

A. Actors and trust assumptions

We consider four actors. The *issuer* is the Lattice runtime that performs a run and signs a receipt with a private key. The *provider* is an external model service that the runtime calls and that we treat as untrusted and possibly buggy. The *verifier* is any party that later receives a receipt and wishes to check it, possibly without trusting the issuer at the time of verification. The *adversary* may sit between these actors or may hold material that looks legitimate.

B. Adversary capabilities

We assume an adversary who can tamper with a stored receipt, a transcript, or a claimed output after a run completes. We assume an adversary who can attempt to substitute a different model route while presenting a receipt that attests to the intended route. We consider an adversary who has obtained a valid signing key and who then tries to mint a receipt in an older schema in order to bypass integrity fields that newer schemas commit to, a schema downgrade attack. We consider an adversary who replays a valid receipt from a different run, a different relying party, or an earlier time and presents it as proof of the current run. We also consider a careless issuer whose run inputs contain secrets or personally identifiable information that must never be embedded in a signed and shareable artifact.

C. Security goals

The runtime targets five goals. First, integrity of the run record: a receipt that has been altered must fail verification. Second, non-repudiation through signature: a verifier with the public key can confirm which key signed a receipt and whether that key is active, retired, or revoked. Third, redaction before signing: personal data matched by a detector, and the tripwire message that would otherwise quote it, are reduced to a detector name, a length, and a mask before the canonical bytes are produced. The signing key and the provider credential are not receipt fields. A free-text field the caller chooses to store is the caller's responsibility. Input and output hashes are unsalted SHA-256, so they hide high-entropy values only: anyone who holds a receipt can confirm a guessed low-entropy value, such as a social security number or a yes or no decision, by hashing candidates. Fourth, determinism for replay: a verifier can recompute the hash of the recorded output and compare it to the value the receipt commits to. Fifth, downgrade resistance: a receipt in the original schema, or below a caller-supplied `minVersion` floor, is rejected before any cryptographic work. Without that floor, a v1.1 receipt still verifies.

D. Non goals

Several properties are explicitly out of scope. We do not provide confidentiality of provider traffic, which is the responsibility of transport security. We do not reproduce the live behavior of a remote model, since model providers are non-deterministic and outside our trust boundary. Replay checks the recorded output against its signed hash; it is not a fresh call to the provider. A signature alone proves integrity and attribution, not freshness: binding a receipt to a relying party, a nonce, or a validity window requires the optional v1.4 fields and the matching verifier options described under verification below, and a verifier that must accept a receipt only once keeps its own record of accepted content identifiers. Dropping leaf receipts from an open receipt tree leaves a smaller tree that still verifies. The chain verifier then returns a `tail-truncation-unchecked` warning unless the caller passes an expected count. A crew run closes its tree instead: its root declares closure and its last receipt lists the content identifiers of every other member, so a dropped leaf or a dropped closer fails verification without an out-of-band count under a verifier that enforces closure (Section V-G). We also do not operate a hosted control plane. Lattice is a runtime SDK, and the receipt is a portable artifact that travels with the run rather than a record locked inside a service.

TABLE I

AGENT RECEIPT SYSTEMS COMPARED WITH LATTICE. THE SIGNER, POLICY, ROUTING, AND MULTI-PROVIDER CELLS SUMMARIZE EACH PAPER'S OWN DESCRIPTION; "NOT STATED" MEANS THE SOURCE DOES NOT SAY. THE OFFLINE-VERIFY AND SIGNED-OUTPUT-BINDING CELLS AGREE WITH THE RUNS IN TABLE VI, WHICH USED A MOCK TRANSPARENCY LOG FOR SELLO AND A MOCK CHAIN ANCHOR FOR AGENT FLIGHT RECORDER. SIGNED OUTPUT BINDING MEANS THE RECORD IS SIGNED AND THE SYSTEM'S OWN VERIFIER REJECTS A BYTE EDIT.

System	Signer	Offline verify	Signed output binding	Policy checks	Routing	Multi-provider
Sello [18]	receiving service, Ed25519 in COSE	yes with a mock log; the paper's verifier needs a log inclusion proof and a key registry	yes; hashes only	no; service denials are recorded	no	not model specific
Agent Flight Recorder [19]	none per event; ECDSA on the chain anchor	yes with a mock anchor; the paper's verifier reads the chain	no; events are hash-chained, not signed	no; verdicts logged	no	not stated
NovaFabric [20]	operator; collector Ed25519 and tenant ECDSA seal	yes for a sealed capsule with a self-asserted key	yes for the sealed capsule; no for the HMAC record	partial; gates on export and replay	no	yes, capture
Janus [21]	log daemon and participants, Ed25519	yes; log and one public key	yes; signed log segments	yes; pre-execution and pre-release gates	no	not stated
Lattice	runtime signer over a key set, Ed25519 in DSSE	yes; public key only	yes; signed output hash	yes; invariants, budgets, and policy at run time	yes; deterministic, selected route signed	yes; seven adapters

IV. DESIGN

Figure 1 shows the end-to-end run lifecycle that produces and then consumes a capability receipt. The subsections that follow walk through each stage.

A. The capability-first run model

A run is described by a small intent: a `task` string, a set of `artifacts` that may be text, JSON, images, audio, documents, or tool results, a map of declared `outputs` with their schemas, a `policy` that expresses privacy, latency, provider allow and deny lists, and budget, and an optional `contract`. The developer does not select a model. The runtime selects one from a capability catalog and records the decision in the execution plan. The receipt signs the selected route next to hashes of the input artifacts, the output, and the contract. The task text and the policy are not signed, so a receipt shows which route served a run and which inputs and outputs it bound, not what the run was asked to do. How an artifact reaches the model depends on the provider adapter (Section V-C).

B. Deterministic and inspectable routing

Routing is implemented as a pure function, `routeDeterministically`, that takes the capability catalog, the request, and the policy and returns a route decision. It performs no input or output, draws no randomness, and never calls a model. The function first applies hard filters that reject a candidate when the provider is unavailable, when an explicit provider or model override does not match, when a required input or output modality is unsupported, when structured output is required but unsupported, or when the estimated input exceeds the context window. It then applies policy filters that reject on an allow or deny list, on a privacy class, on a no-logging or no-upload requirement, on a latency class, or on a budget ceiling. Surviving candidates are scored from cost, latency, and context headroom, and ties are broken by a stable order over acceptance status, score, provider name, and model name, with names compared by code unit rather than by locale so the order is the same on every machine. The catalog position is deliberately not part of the score, so the decision does not depend on the order in which providers were registered; a test evaluates every permutation of a near-equal-cost catalog and requires the same selected route and the same fallback chain. The decision exposes the selected route, all candidates, all rejected candidates, a fallback chain of policy-preserving alternatives, and a deduplicated list of typed reasons when no route is possible. The full execution plan is stable JSON with a nine-stage pipeline: analysis, transforms, context packing, provider packaging, tool execution, execution, validation, tripwire, and persistence. The execution plan returns this decision in full. The receipt signs only the selected route, as a provider identifier, a capability identifier, and an attempt number, and not the candidates, the scores, or the policy, so it attests to which route served the run but not to why that route was chosen. Because the router is a pure function, a reviewer who holds the same catalog, request, and policy can reproduce the decision by re-running it.

C. Output invariants with terminal semantics

Lattice lets a developer attach output invariants through a fluent builder, `inv`, with constructors `mustCite`, `fieldFromTable`, `noPII`, and `matches`, where the last validates against a Standard Schema [26]. These invariants are evaluated by a pure kernel, `evaluateTripwires`, that performs no input or output, reads no clock, and uses no randomness, so the same output and the same invariants always yield the same result. The kernel aborts on the first violation. It scans strings and walks nested objects, arrays, and wildcard paths in a deterministic order, so a value buried inside a structured output is not missed.

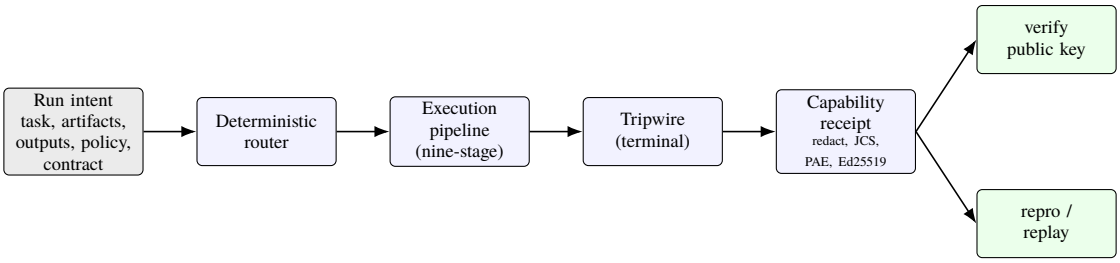


Fig. 1. The Lattice run lifecycle. A run intent is routed by a deterministic and inspectable function, executed through a nine-stage pipeline, checked by terminal output invariants, and committed to a signed capability receipt. The receipt is later consumed by offline verification and by the `repro` check of a kept output against the signed output hash.

The default `noPII` detectors are regular expressions for four formats. An email address matches as written, and so do a dashed United States social security number (`ddd-dd-dddd`) and a separated ten-digit United States phone number. A spaced or undashed nine-digit number counts as a social security number only when an SSN cue such as `ssn` or `social security` is the nearest preceding word. A plain ten-digit run counts as a phone number only when it is a valid North American number that follows a phone cue such as `phone` or `call`, or a `+1` or `1-` trunk prefix. A card number is 13 to 19 digits, optionally grouped by single spaces or dashes, that pass the Luhn checksum; runs that start with 0 or 1, and 13-digit runs that start with an ISBN-13 prefix, are skipped. A social security, phone, or card candidate is also skipped when the nearest preceding word on the same line is an identifier label: order, invoice, sku, ref, po, isbn, txn, or one of a few abbreviations. A phone number with a trunk prefix is flagged even after a label, and labels do not affect email addresses. Ordinary words such as customer, account, ticket, or id are not labels. These rules trade recall against precision in known ways. A real value right after a label ("ref 123-45-6789") is missed, and so is a bare nine- or ten-digit value without its cue. A value under a telling object key, such as `{ssn: "123456789"}`, is missed because the detectors read string values, not field names. In the other direction, about one random long digit run in ten passes the Luhn check, so a text that lists k long numeric identifiers is flagged with probability near $1 - 0.9^k$. Names, street addresses, and non-United States formats are not detected.

By default `mustCite(name)` is a structured check. The output's top-level `citations` list, or else its `evidence` list, must contain the artifact name as a string entry or as an object entry whose `source` key, or another configured key, equals it. When neither top-level list names the artifact, nested `citations` and `evidence` lists are searched as well, unless the invariant declares a path to the list. The default never reads prose. Three options extend it. With `inline`, a bracket or parenthesis group whose content, or one comma- or semicolon-separated part of it, equals the name counts when no located list has entries; code spans and link text are ignored. With `rejectUncited`, a list entry that names the artifact is rejected when the prose carries square-bracket citations, such as `[1]`, `[Smith 1999]`, or a list entry's identifier, and none of them resolves to the artifact. `textPath` limits both scans to one field. Each option enters the contract declaration only when set, so an existing contract keeps its hash. The check stays syntactic. By default an answer passes when its list names the artifact even if its text cites nothing. `inline` also accepts a marker in a sentence that disowns the source. `rejectUncited` reads only square brackets, so a parenthetical citation of another work goes unseen, and without `textPath` a bracketed number in an unrelated string, such as a log line, counts as a citation. No option judges whether the cited artifact supports the claim.

When a `noPII` invariant fires, the runtime error returned to the caller carries the detector name and the matched substring so the violation can be debugged, and the receipt redaction policy replaces that substring with the detector name, the match length, and a shape mask before anything is signed; the other three invariant kinds have their observed value reduced to a type label from a closed set in the same step, and every kind's message is rebuilt from the invariant kind and the declared path so that neither model produced object keys nor schema issue text reach the signed bytes. A violation produces a `TripwireViolationError` that carries the flag `terminal: true`. The predicate `isTerminal` returns true for a tripwire violation and for a no contract match, and the fallback chain consults this predicate so that a terminal failure is never retried. This matters for safety: a system that retries until a banned output passes is worse than a system that fails closed, and terminal semantics make failing closed the default.

D. Capability contracts and preflight

A contract expresses requirements that must hold before and around a run: a budget with a maximum cost and, for agent runs, iteration and wall time caps, a set of invariants, a quality floor, a set of required modalities, and a required privacy class. A pure preflight evaluator, `evaluateContractAgainstRoute`, runs inside the router and can reject a route with a typed reason such as a budget exceeded, a missing modality, or a privacy mismatch. A quality floor reason is reserved for the evaluation tool described later. Usage is normalized to prompt tokens, completion tokens, and a cost in United States dollars on every outcome, success or failure, and a null cost is distinct from a zero cost so that an unmeasured price is never confused with a free call.

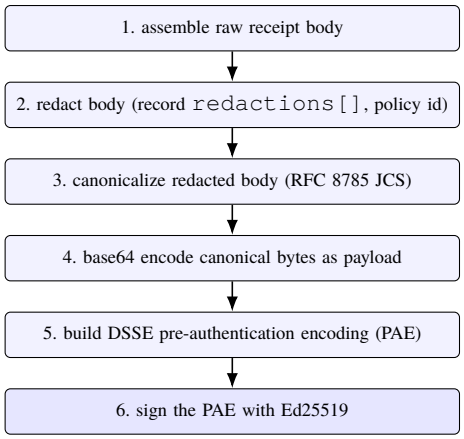


Fig. 2. The receipt construction pipeline. Canonicalization is only ever applied to the output of redaction, so signing unredacted bytes is structurally impossible.

E. The capability receipt

The receipt is the central artifact. Its body records the requested and observed model, the selected route, normalized usage, the contract verdict, the contract hash (null without a contract), SHA-256 hashes of the input artifacts, the output hash, a lineage root over the artifacts, and the redaction manifest, with optional audience, nonce, expiry, parent, and chain-closure fields. The task text, the packed prompt, and the policy are not part of it. Its construction enforces a strict ordering that is the heart of the security argument. Figure 2 shows the pipeline, and Figure 3 shows the resulting envelope.

First, the runtime assembles the raw receipt body from the run record. Second, it redacts the body. Redaction rewrites the tripwire evidence for every invariant kind, replacing a matched personally identifiable value with a detector name, a length, and a shape mask and reducing other observed values to a type label, and records a manifest of redactions, each with a path and a reason, and a signed redaction policy identifier whose default value is `lattice.default.v1`. Each manifest entry is written at the moment of the corresponding rewrite, so the manifest cannot claim a redaction that did not happen. Third, it canonicalizes the redacted body with RFC 8785 [11]. Fourth, it base64 encodes the canonical bytes as the payload. Fifth, it builds the DSSE pre authentication encoding over the payload type and the payload. Sixth, it signs the pre-authentication encoding with Ed25519. The ordering is enforced structurally, not by convention: the canonicalization function is only ever called on the output of the redaction function, so a code path that signs unredacted bytes is unrepresentable. This is what we mean by redact-then-sign as an architectural invariant rather than a guideline.

The signing key is identified by a `kid` and resolved through a key set. A key set maps a `kid` to a key entry whose state is *active*, *retired*, or *revoked*. Cost is serialized as an I-JSON [12] string or null so that receipts never carry raw floating point numbers. The signature is computed over the pre-authentication encoding rather than over the raw JSON, which removes any ambiguity about what the signature commits to.

```
{
  "payloadType":
    "application/vnd.lattice.receipt+json",
  "payload": "<base64(JCS(redacted body))>",
  "signatures": [
    { "keyid": "<kid>",
      "sig": "<base64(Ed25519(PAE))>" }
  ]
}
```

Fig. 3. The Dead Simple Signing Envelope produced for a receipt. The pre authentication encoding concatenates the literal `DSSEv1`, the payload type and its length, and the payload and its length, all in a fixed order.

F. Schema versioning and downgrade defense

Receipts carry an explicit version. Five versions exist: `lattice-receipt/v1`; v1.1, which adds the step-marker fields; v1.2, which adds an optional and signed model class field recording the training class of the chosen model; v1.3, which adds the parent receipt content identifier and the artifact lineage root; and v1.4, which adds optional audience, nonce, and expiry fields for freshness binding. Two further optional v1.4 fields carry chain closure (Section V-G). New receipts are minted at v1.4 and verifiers accept v1.1 and later. Verification rejects a body whose version is the original v1 or is absent, with the error `schema-version-too-low`, and it does so before any cryptographic work. A verifier that depends on a newer schema's commitments raises the floor with a minimum version option and receives the same error for anything older. This is the downgrade defense from the threat model. An adversary who holds a valid signing key still cannot mint a v1-shaped body to evade the integrity fields that later schemas commit to, because the structural version check fails first.

G. Verification

Verification, `verifyReceipt`, returns a result that is either an ok value with the verified body and the resolved key state, or a failure value with a typed error. There are exactly ten error kinds, listed in Table II. Verification checks the envelope shape, the schema version against the floor, the key existence and state, the re-canonicalization match, and the Ed25519 signature, with a defense in depth check that the `kid` inside the body matches the key entry. Only after the signature holds does it apply the optional binding checks: an expected audience, an expected nonce, the receipt's own expiry instant, and a maximum age over the issue time. Running the binding checks last means an unauthenticated envelope cannot be used to probe which audience or nonce a verifier expects. A companion function, `verifyChain`, verifies a set of receipts as a tree linked by parent content identifiers: every link must verify, content identifiers must be unique, all receipts must share one run identifier, exactly one receipt may lack a parent, and every receipt must be reachable from that root. Those chain failures are a separate `VerifyChainErrorKind` union (an invalid link, no root, multiple roots, an orphan link, a run-identifier mismatch, a duplicate content identifier, a count mismatch, or a root mismatch). They are not among the ten

TABLE II
THE TEN VERIFYRECEIPT ERROR KINDS, IN THE ORDER THEY ARE CHECKED.

Error kind	Fires when
envelope-malformed	the envelope cannot be decoded or has no signatures
version-mismatch	the body shape, version, or closure field is unknown or malformed
schema-version-too-low	the body is v1, has no version, or is below the caller's floor
key-not-found	the kid is not in the key set
key-revoked	the resolved key is in the revoked state
canonicalization-mismatch	the re-canonicalized body differs from the signed bytes
signature-invalid	the Ed25519 check fails or the body kid mismatches
audience-mismatch	the verifier expects an audience the receipt lacks or differs on
nonce-mismatch	the verifier expects a nonce the receipt lacks or differs on
receipt-expired	the receipt's expiry instant or the verifier's maximum age has passed

VerifyErrorKind values in Table II. When the root declares closure, the closing receipt's member list must equal the other presented content identifiers, so a dropped or added member is a count mismatch and a malformed closer is an invalid link. Two options adjust this: requireClosure fails a chain that declares no closure, and partial verifies a disclosed subset, which must include the closer and be listed by it. A successful result carries structured advisories, each a code, a severity of warn or info, and a message, for checks the caller left out: for example, an audience on the receipt that was not checked, or an open chain verified without an expected count. receiptVerifyAdvisories returns the same records for a single receipt.

H. Offline verifiable replay

A receipt becomes useful for reproduction through replay. A replay envelope carries optional receipt and contract fields. The function materializeReplayEnvelope reconstructs a replay envelope from a receipt and a set of content-addressed artifacts, and it enforces a critical ordering: it verifies the receipt before it loads any artifact. A test confirms that the artifact loader is called zero times when verification fails, which prevents a tampered receipt from causing artifact loads or side effects. Materialization can fail with one of three kinds: verify-failed, artifact-load-failed, and envelope-malformed. A property test asserts that the round trip from createReceipt through materializeReplayEnvelope through replayOffline preserves the output hash, which is the concrete statement of the determinism goal. replayOffline returns the outputs the caller supplies and calls no model, so replay checks that a kept output is the one the receipt signed. It does not re-run the model.

I. The four guarantees and their enforcement

Table III maps each guarantee to the mechanism that provides it and to the property or test that enforces it. The point of the table is that none of the guarantees rest on a convention or a comment. Each is enforced by structure or checked by a test that fails the build when the property is violated.

TABLE III
EACH GUARANTEE, ITS MECHANISM, AND ITS ENFORCEMENT.

Guarantee	Mechanism	Enforcement
Deterministic routing	a pure routing function	recorded decision, no I/O or randomness
Terminal invariants	isTerminal consulted by fallback	fallback never retries a terminal error
Redact-then-sign	canonicalize only on redaction output	ordering is structurally unrepresentable otherwise
Verifiable replay	verify before load	loader call count is zero on verify failure; output hash round trip

V. IMPLEMENTATION

A. Packaging and footprint

Lattice publishes a core runtime package and a command-line package. The core package has a deliberately small runtime dependency surface of four packages: canonicalize for RFC 8785 encoding, @standard-schema/spec for schema contracts, mime for artifact typing, and @noble/ed25519 for one audit signing path. The schema library used in tests is a development dependency only, so a consumer of the core package does not inherit it. Modular public subpaths exist for providers, audit, context, artifacts, routing, tools, storage, eval, agents, and core. These entry-points let a downstream runtime adopt only the provider layer, only signed audit artifacts, or only context and artifact preparation. Package names, the publication workflow, and the Node version used for the measurements are in Appendix XI.

B. Signing internals

Signing uses the Node 24 WebCrypto interface with the literal algorithm name Ed25519. Keys are represented as JSON Web Keys of the octet key pair type and signatures are 64 bytes as specified by EdDSA [13]. To guard against a silent regression in the platform implementation, a development-only parity oracle signs the same message with an independent Ed25519 library and asserts a byte-for-byte match in the test suite. The oracle is never on the production signing path.

C. The seven-provider parity contract

Lattice defines seven logical provider adapters: openai, openai-compatible, anthropic, gemini, xai, open-router, and lm-studio. An internal invariant named INV-03 states that every improvement must work equally across all seven universal provider targets. A parity smoke test makes this concrete. As summarized in Table IV, the test iterates all seven adapters against provider-shaped fake responses and asserts a uniform adapter shape, string raw outputs, the normalized usage shape, an error on a non-success response, propagation of an abort signal into the underlying fetch, a raw response pass through, and that the set of adapter identifiers has size seven. Parity gives every adapter the same receipt format and the

TABLE IV
WHAT THE INV-03 PARITY SMOKE TEST ASSERTS ACROSS ALL SEVEN PROVIDERS.

Property	Assertion
adapter shape	each adapter exposes the same surface
raw outputs	content is returned as strings keyed by output name
usage shape	normalized prompt and completion tokens and cost
error path	a non-success response throws with provider identity
cancellation	the request abort signal reaches the fetch
raw response	the original parsed body is available
identity	the set of adapter ids has size seven

same verification path. It does not make the adapters transmit the same request. The `openai`, `openai-compatible`, `xai`, `openrouter`, and `lm-studio` adapters share one request builder that sends each artifact as a text part holding JSON and includes the artifact's value only when it is a string. A data-URL string is inlined as text, and other values, such as binary image bytes and JSON objects, are dropped, yet the receipt still hashes the dropped input. Its input hashes therefore record what the caller supplied, not what the provider received. The `anthropic` and `gemini` adapters send native image parts. The adapters also differ in sampling parameters: `anthropic` sends a fixed token cap, `gemini` sends a fixed temperature and token cap, and the OpenAI-shaped adapters send neither. Across adapters a receipt means the same thing about its signed fields, but not about what the model saw.

The same provider contract exposes opt-in native provider features for provider-only consumers. `ProviderRunRequest.nativeTools`, `nativeToolChoice`, and `nativeStructuredOutput` let an external orchestrator use native tool calling and structured output where adapters support it, without changing the default `ai.run` or agent behavior.

D. The capability registry

Routing consults a capability catalog backed by a registry of 332 model profiles. Of these, 328 are generated from a snapshot of an OpenRouter model feed and 4 are hand maintained static supplements for direct adapters, namely an Anthropic Opus profile, a Gemini Pro profile, an LM Studio local template, and an xAI Grok profile. A weekly continuous integration drift gate guards the generated file so that the catalog does not silently diverge from the upstream feed. At call time the runtime can refine a profile through capability negotiation, fetching a live model listing with a time to live cache and inflight request coalescing, and falling back to the registry on a transient error. Adapter quirk flags capture provider-specific behavior so that the runtime can adapt without breaking the parity contract.

E. External audit and standalone adoption

The receipt layer can also be used without Lattice execution. The `createExternalExecutionAudit` helper accepts externally produced request, response, output, usage, and verdict data, then mints the same signed receipt family, sidecar, replay envelope, and raw request, raw response, input, and output hashes that native Lattice runs produce. Failed verdicts preserve sidecar and receipt evidence but omit replayable outputs, so a failed external execution remains auditable without becoming an offline success fixture.

For runtimes that want preparation without provider execution, `prepareCoreRun` returns artifact references, optional storage references, a context pack, advisory routes, input hashes, warnings, and an execution plan. The tools subpath also exposes MCP resource, prompt, and tool-result artifact helpers, so tool systems can convert their outputs into replayable artifacts without importing the agent runtime.

F. The command-line tool

The `lattice` command-line tool ships in the second package. It uses a lazily loaded subcommand structure. Status codes are listed in Appendix XI. The `verify` subcommand checks the signature and structure of a receipt and prints the key identifier and contract verdict. The `repro` subcommand verifies a receipt, materializes the replay envelope, replays offline, and diffs the recomputed output hash. The `eval` subcommand walks receipt directories or agent snapshots and gates determinism, cost, and quality regressions relative to a baseline, with a disk-backed judge cache. The `receipt diff` subcommand compares signed receipt envelopes and reports structural differences, including the chain-closure fields, and `receipt verify-chain` verifies a receipt tree. Together these commands turn a receipt from a passive record into an executable regression and audit gate.

G. Agents and chained receipts

Lattice exposes a runtime agent entry point, `ai.runAgent`, that drives a uniform tool-use loop across all seven providers, and an opt-in multi-agent crew surface, `defineAgent` and `runAgentCrew`. `ai.runAgent` returns no receipt. The loop does not use the signer passed to `createAI`, and with a signer on the agent intent it mints per-iteration checkpoint receipts into tracer events only, so a caller without a tracer keeps none. A crew shares a budget with structural caps on total iterations, per-agent iterations, concurrent children, and depth, and it coordinates rate limits through a token bucket with default limits.

With a signer, a crew returns a crew-start root, a completion receipt for the root agent and for each child run that succeeded, and a crew-close receipt, so a crew of N agents that each run once returns $N + 2$ receipts when every child run succeeds. A child whose run fails gets no completion receipt. The crew-start receipt hashes the tree of agent identifiers, not their intents, tools, or policy, and declares closure. Each completion receipt chains to the root through a parent receipt content identifier and commits to that agent's output hash. The crew-close receipt lists the sorted content identifiers of every other member. The crew budget verdict is fixed when agent work ends, before the root agent's completion receipt and the crew-close receipt are signed, so signing those two receipts cannot

push a crew over budget, and the crew-close receipt signs a failed verdict when the crew exceeded its budget. The crew-start and child completion receipts are signed during the run and count against its wall time. A mint failure after the crew-start receipt follows the configured signer-error policy: by default the result lists each failed mint and a configured tracer receives an event, and the run can instead be set to throw. The closing list names only the receipts that were minted, so a missing completion receipt still leaves a verifiable chain, while a missing closer leaves the closed chain unverifiable. `verifyChain`, exposed in the command-line tool as `receipt verify-chain`, checks every link, the single root, the parent linkage of the whole tree, and the closing list. Verifiers in the 1.6.x releases accept closed chains but do not enforce closure. An optional expected count still covers open chains, and an optional expected root identifier detects a replaced root. A helper, `evalAgentRun`, gates iterations to goal and cost against a baseline.

Three runnable examples exercise these surfaces end to end: a support work inbox that demonstrates a contract, a signed receipt, a tripwire violation, and a no contract match; a single agent loop that mints per-iteration checkpoint receipts into tracer events; and a crew example that demonstrates chained receipts and an evaluation gate.

VI. Evaluation

We evaluate Lattice in two ways. The first is measured: the cost that receipts add to a run, the size of a receipt, whether the verifier rejects every tampered receipt, and whether every kept output matches its signed hash, all against a mocked provider so that no model behavior enters that measurement. The second is assurance by construction: the guarantees are encoded as invariants and property tests that fail the build when a guarantee weakens. Output quality depends on providers outside our trust boundary. Section VI-C records a local open-weight model and wraps the same recorded runs with the other record systems. Section VI-D adds free hosted models (Tables VII and VIII, and Figures 9, 10, and 11), including Lattice processing time separated from provider latency and rate-limit backoff. It also reports signed output binding (Figure 12), tamper classes across systems (Figure 13), the capabilities observed on these workloads (Figure 14), the `noPII` detectors against other detectors on public data (Tables IX and X), and local wrappers around the same model (Table XI and Figure 15). Those runs are not part of the mocked table.

A. Measured cost and tamper resistance

A benchmark script in the repository (`scripts/bench-receipts.mjs`) runs against the built package through its public exports with the in-repo mocked provider and no network, and writes every number below to a committed JSON file (`paper/bench/results.json`) so the table can be regenerated on any machine. Table V reports that run. The Node version and processor are in Appendix XI. This benchmark is separate from the offline harness in `eval/`, which runs its own tamper suite.

TABLE V
Measured receipt cost and tamper resistance against a mocked provider. Latencies are wall clock; n is the number of samples.

Measurement	Result
Create a receipt: redact, canonicalize, sign	median 0.067 ms, p95 0.095 ms (n=1,000)
Verify a receipt	median 0.126 ms, p95 0.177 ms (n=1,000)
Receipt issuance overhead on a run with a 1 MiB image input	1.402 ms, median of 40 signed runs minus median of 40 unsigned runs
Envelope size	993 bytes with no inputs, plus 89 bytes per input hash (signed body 593 bytes, plus 66.875)
Tamper sweep	3,744 of 3,744 mutations rejected: 72 receipts, 52 mutations each across 14 classes
Offline replay: kept output against the signed output hash	350 of 350 runs match

Creating a receipt takes a median of 0.067 ms: the body is small and Ed25519 signing is fast. Verification takes a median of 0.126 ms; besides the signature, it re-canonicalizes the body and checks the ten failure branches in order. With a 1 MiB image on the run, the median signed run takes 1.402 ms longer than the median unsigned run. The signed path adds SHA-256 hashing of the image bytes, for the input hash and the lineage root, and receipt creation. A receipt with no inputs is 993 bytes on the wire, and each input hash adds about 89 bytes. The tamper sweep applies 52 mutations to each of 72 signed receipts: 30 single-bit flips at random positions in the payload, 10 in the signature, an unknown key identifier, a valid but wrong key identifier, a swapped body key identifier, a downgrade to the original schema version, a removed version, a dropped output hash, an edited audience, an edited nonce, an extended expiry, reordered keys in the payload, a base64url payload, and a payload swapped in from another valid receipt. Every mutation is rejected by a typed verification failure, and none makes the verifier throw. The replay check runs 350 runs with distinct outputs through the runtime with a signer, materializes each receipt through the artifact loader with the kept outputs, replays it offline, and compares the recomputed output hash with the signed one; all 350 match.

B. Guarantees also encoded as tests

The measurements above are the evidence for cost, tamper resistance, and replay. The repository also contains tests that fail the build if selected guarantees regress. A test count is not itself a measurement, so this subsection only names what those tests check.

We organize those checks around the four guarantees.

Determinism of routing and invariants. The router and the tripwire kernel are pure functions. Tests assert that the same inputs always produce the same route decision and the same tripwire result, that every permutation of a near-equal cost catalog yields the same selected route and fallback chain, and that the no personally identifiable information walk reports the same violation regardless of key insertion order; the absence of

input, output, clock reads, and randomness in these functions is verified by construction and by review. Because the decision is recorded in the execution plan, a reviewer can reproduce a route by re-running the function.

Redact-then-sign. The ordering is structurally enforced because the canonicalization function is only ever called on the output of the redaction function. Tests confirm that the redaction manifest and the signed redaction policy identifier appear in the signed body, that for every invariant kind a sentinel value placed in the evidence does not appear anywhere in the canonical signed bytes, that a no personally identifiable information violation is signed as a detector name, a length, and a shape mask while the runtime error still carries the match, and that redaction is idempotent.

Downgrade resistance and verification. Tests cover all ten `VerifyErrorKind` values, including audience, nonce, and expiry mismatches and the caller-raised version floor. Chain failures are not among those ten. They use `VerifyChainErrorKind`. In particular, a body in the original v1 schema, or a body with no version, is rejected with a schema version too low error before any cryptographic work, which demonstrates the downgrade defense. Tests also confirm that a revoked key yields a key revoked error and that a mismatch between the body key identifier and the key entry yields a signature invalid error.

Verifiable replay. A test asserts that the artifact loader is called zero times when receipt verification fails during materialization, which shows that a tampered receipt cannot trigger artifact loads. A property test asserts that the round trip from receipt creation through materialization through offline replay preserves the output hash, which is the determinism goal stated as a checkable property. The `repro` subcommand exercises the same path end to end and returns a distinct exit code for a match and for a drift.

Parity. The INV-03 smoke test exercises all seven providers and asserts the uniform behavior summarized in Table IV, including that the set of adapter identifiers has size seven. Parity is what lets a single receipt format and a single verification path apply regardless of which provider served the run.

A change that weakens one of these checks fails the suite. The checks keep the guarantees from drifting. They do not replace the measured results.

C. Local open-weight runs and other record systems

The artificial evaluation mocks the provider. This subsection does not. Ollama 0.40.7 served `llama3.2:3b` (Llama 3.2, 3.2B, Q4_K_M, digest `a80c4f17acd5`, 2.0 GB) on the same host as the other measurements (Node v24.22.1, 4 vCPU, 15.63 GB RAM, no GPU). Lattice called it through the OpenAI-compatible adapter with no API key and an explicit base URL ending in `/v1`. `createOpenAIProvider`, whose default base URL builds a chat URL without `/v1`, was not exercised. The adapter does not send temperature or a token cap, so a fetch wrapper added both to every request. The Lattice numbers are therefore for a request the adapter does not build on its own. No vision model was loaded. No hosted provider key was present.

On the one-line prompt "Reply with the single word pong," temperature 0, five paired trials, the median plain HTTP call was 278 ms. A signed Lattice run of the same task was 559 ms and an unsigned Lattice run was 745 ms (Figure 4). The first plain call, 4845 ms, is a cold start and pulls the mean up; the median does not. The signed median is lower than the unsigned median. That gap is generation noise, not a claim that signing is faster. What is stable is the time spent outside the provider HTTP call: 0.439% of end-to-end latency at the signed median (minimum 0.335%, maximum 2.469%). The Lattice prompt body was 382 characters against a one-line plain prompt, so most of the extra wall time is a larger prefill, not the signature. The short signed receipt was 996 bytes.

A refund-policy question with a `mustCite` invariant passed on all three temperature-0 runs and all three temperature-0.7 runs. These verdicts come from the `mustCite` rules at commit `a8cc091`, where an inline `[name]` or `(name)` marker containing the artifact name counted as a citation by default. The current default reads only a structured citation list, so these answers, which cite the clause only inline, would fail it unless `inline: true` is passed. Each temperature produced one identical answer across its three trials (n=3). The two temperatures did not produce the same sentence. All 6 signed receipts verified. For each, the signed output hash matched the kept answer and changed after a one-character edit (6/6). This checks the output binding, not a re-run: `replayOffline` returns the outputs the caller supplies. A synthetic support ticket containing an email, a US phone number, and a dashed SSN was flagged by those three detectors. The model's answer contained none of them, the `noPII` invariant passed, and the signed receipt did not contain the planted strings. A two-step agent called `lookup_policy` once and answered; the returned agent object carried no receipt. A two-agent crew completed in 3 iterations with 4 receipts.

The public sets are harder than the synthetic harness, and the gap is the result (Figure 5). ai4privacy/pii-masking-300k [27] English validation (revision `c8c77895a005`, 7,946 documents; academic, non-commercial, no redistribution) gives email precision 0.955 and recall 0.979 (1533 documents with a gold email). US SSN precision is 0.325 and recall 0.436. US phone precision is 0.316 and recall 0.333. Of the SSN false positives, 1,074 overlap a passport span: a 9-digit passport is the same shape as an SSN, and the detector cannot tell them apart. Of the phone false positives, 629 overlap a social-security span. There is no gold credit-card span in this file; the card detector fired on 7 documents anyway. Among 1,070 US-shaped SSN spans, the single match the API returns overlaps 663. It misses 370 because that one match fell on a different number earlier in the document, and 37 because it matched nothing. Gretel English finance test [28] (Apache-2.0, revision `7b844d167385`, 2,891 rows) is the same pattern at smaller n: email recall 0.934, SSN precision 0.161. ASQA dev [29] (Apache-2.0, revision `084060f16b46`, 948 answers) is a loss

for `mustCite`. A gold Wikipedia title occurs as a substring in 525 answers, but the invariant accepts at least one gold title in 7. Only 2 answers use a square-bracket citation. The `mustCite` count was measured at commit `a8cc091`, where an inline `[name]` or `(name)` marker containing the title counted as a citation by default. The current default never reads the answer text, and these answers carry no citation list, so it accepts a gold title in none of them. The opt-in `inline: true` counts only a bracket or parenthesis whose content, or one comma- or semicolon-separated part of it, equals the title. The check is syntactic. It does not score whether a citation supports the claim. These dataset rows were measured with the `noPII` detectors at commit `a8cc091`. The current detectors differ: an identifier label suppresses a match only when it is the nearest token, and bare 9-digit, spaced, and 10-digit numbers need a type-specific cue such as SSN or phone. The artificial evaluation covers the current detectors and the current `mustCite` rules.

Table VI wraps the same recorded request and response bytes. Lattice's receipt and Sello's encrypted receipt (mock transparency log, not the public log) omit the planted strings. An AFR log with disclosure encryption also omits them; the default AFR log contains them. Janus's local evidence appender stores the payload in the clear and the byte edit fails `verify.SegmentDir`. A sealed NovaFabric capsule (self-asserted ECDSA key, no RFC 3161 timestamp) fails `nova verify` after a newline is appended to stdout, and the capsule contains the planted text. `nova validate` on that edited capsule still exits 0. cosign `sign-blob` [30] with a local key and no Fulcio certificate detects a byte edit and signs the plaintext log. That run kept cosign's default transparency-log upload, so it wrote an entry to the public Rekor log. Its 4,916 ms wrap time includes that round trip, and `verify-blob` checked the log online. in-toto's DSSE link [6] detects a byte edit and does not embed the plaintext. A hash chain detects an edit that leaves the stored hashes untouched and accepts a rebuilt chain, because nothing outside the file anchors the head. NovaFabric's HMAC record has the same shape with a shared secret: an edit fails, and a re-sign with that secret passes. OpenTelemetry GenAI spans and a plain JSON log have no integrity check; both store the transcript. TrustTrack has no implementation we could run. Langfuse was not started: this host has no Docker daemon. Lattice, the NovaFabric capsule, cosign, in-toto, Sello, and Janus sign the record and reject a byte edit; Table VI lists that as signed output binding.

The wrap times are single cold samples with different scopes, so they order the systems only roughly. The Lattice total of 5.923 ms is three `createReceipt` calls, and the first call is 5.584 ms of it; Sello's first call is 7.259 of its 9.336 ms. The byte column mixes scopes too (Figure 6): the JSON log, the hash chain, OTel spans, AFR, the NovaFabric HMAC record, the NovaFabric capsule, cosign, and Janus keep the plaintext transcript, while Lattice, AFR disclosure, in-toto, and Sello keep only digests or ciphertext.

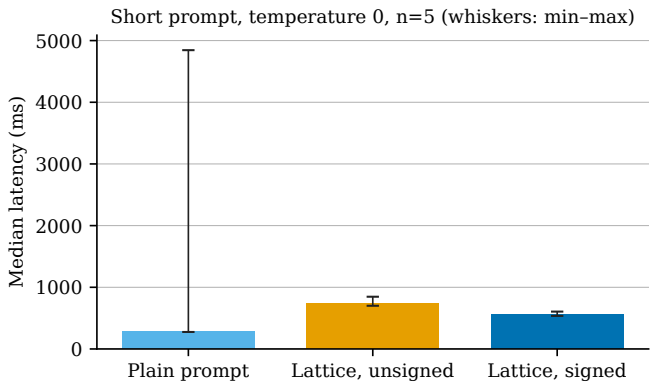


Fig. 4. Median latency of a one-line completion. Whiskers run from the minimum to the maximum of five trials. The plain arm sends only that line. The Lattice arms send the adapter's packed prompt.

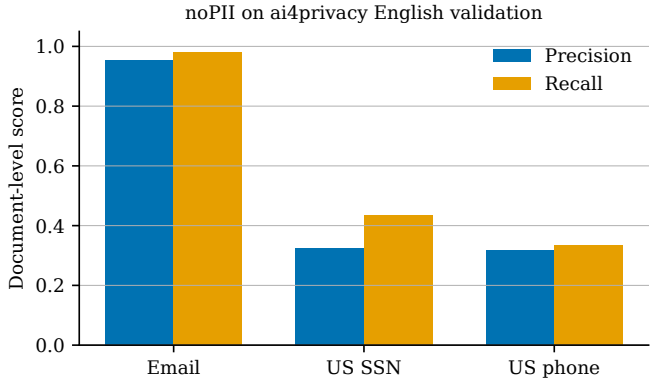


Fig. 5. Document-level precision and recall of the default `noPII` detectors at commit `a8cc091` on ai4privacy English validation. US SSN and US phone lose to identifier types that share their shape.

D. Broader comparison

This subsection adds the detector, citation, gateway, and hosted-model measurements from the same machine.

Lattice's OpenAI-compatible adapter called `https://openrouter.ai/api/v1`. The catalog check found 8 of the requested `:free` ids, and the listed prompt and completion prices were 0. Completed model rows: 8. Upstream HTTP 429 responses were retried. Pacing sleeps, that backoff, and the HTTP time of a failed attempt are excluded from Lattice time and from end-to-end time, so end-to-end time is Lattice processing plus the provider time of the attempt that returned. Provider time includes waiting for the response headers and reading the body. Completion tokens are capped at 8192. The adapter sends no temperature, `max_tokens`, or reasoning field. A fetch wrapper adds temperature and the token cap to every request, and adds `reasoning.effort=low` for models whose catalog lists a reasoning parameter. A 400 that rejects that field is retried once without it. The Lattice rows therefore describe requests the adapter does not build on its own. Table VII uses the provider string returned on the chat response. A drift cell is distinct successful answers, successful

TABLE VI

SAME RECORDED OLLAMA RUNS, WRAPPED BY EACH SYSTEM THAT RAN. WRAP TIME EXCLUDES MODEL GENERATION. WRAP MS IS ONE COLD RUN PER SYSTEM, NOT A WARMED MEDIAN, AND THE SCOPES DIFFER: LATTICE AND SELLO ARE IN-PROCESS JAVASCRIPT CALLS SUMMED OVER THREE RECORDS, COSIGN AND THE NOVAFABRIC CAPSULE ARE ONE CLI INVOCATION EACH, INCLUDING PROCESS START AND KEY LOADING, JANUS REPORTS WHOLE MILLISECONDS FROM ITS OWN TIMER, AND THE JSON LOG TIME IS SERIALIZATION ONLY. PII IN ARTIFACT REFLECTS WHAT THE HARNESS GAVE EACH SYSTEM; WHERE IT IS YES, THE BYTE COUNT INCLUDES THE PLAINTEXT TRANSCRIPT. SIGNED OUTPUT BINDING IS YES WHEN THE RECORD IS SIGNED AND THE SYSTEM'S OWN VERIFIER REJECTED A BYTE EDIT. TRUSTTRACK AND LANGFUSE WERE NOT EXECUTED. THE COSIGN RUN UPLOADED TO THE PUBLIC REKOR LOG AND VERIFIED ONLINE.

System	Status	Wrap ms	Bytes	Offline verify	Byte edit detected	Key assumption	PII in artifact	Signed output binding
Lattice receipt	measured	5.923	3210	yes	yes	other key rejected	no	yes
JSON log	baseline	0.033	1045	no	no	—	yes	no
Hash chain	baseline	0.082	1531	yes	yes	rebuild accepted	yes	no
OTel GenAI spans	measured	0.228	1366	no	no	—	yes	no
Agent Flight Recorder	measured	0.797	2328	yes	yes	mock anchor only	yes	no
AFR disclosure	measured	1.907	3357	yes	yes	mock anchor only	no	no
NovaFabric HMAC	measured	0.077	1432	yes	yes	shared secret re-signs	yes	no
NovaFabric capsule	measured	1373.053	21288	yes	yes	self-asserted key	yes	yes
cosign sign-blob	measured	4916.329	1319	online	yes	local key, Rekor upload	yes	yes
in-toto link	measured	34.149	888	yes	yes	functionary key	no	yes
Sello receipt	measured	9.336	1359	yes	yes	mock log	no	yes
Janus evidence log	measured	5	1723	yes	yes	writer key	yes	yes
TrustTrack	paper only	—	—	—	—	—	—	—
Langfuse	not run	—	—	—	—	—	—	—

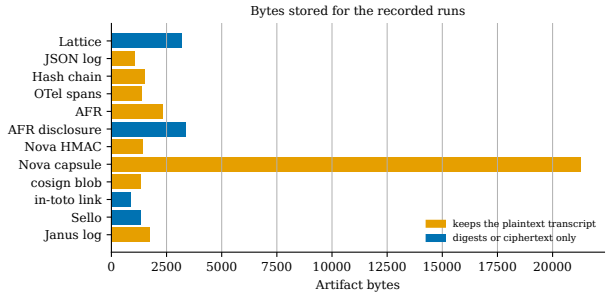


Fig. 6. Bytes retained for the recorded runs. Orange records keep the plaintext transcript; blue records keep only digests or ciphertext. The NovaFabric bar is one sealed capsule, not three hash receipts. AFR disclosure is the encrypted archive. The cosign bar is the blob, the signature, and the public key.

calls, and attempts. A RAG call succeeds when the run returns an answer that passes `mustCite`. The recorded verdicts come from the rules at commit `a8cc091`, where an inline `[name]` or `(name)` marker containing the artifact name counted as a citation by default. The current default reads only a structured citation list and would fail every recorded RAG answer unless `inline: true` is passed. Output binding uses only calls that returned a receipt. The local row is the Ollama run in Section VI-C (drift n=3, overhead n=5). Hosted drift used n=10 when the call returned. On the local Ollama run, the median time outside the HTTP call was 2.4 ms (p95 15.0 ms, 0.439% of that run's signed wall time). That localhost call returns the body with the headers, so the gap is processing time. Across hosted models with a successful signed short prompt, the median Lattice processing time was 1.10–1.69 ms, or 0.032–0.218% of end-to-end time. Table VIII gives the median and p95 in milliseconds and as a percentage. The percentages are recomputed from the per-trial phase times; the `signedLatticePct` field stored in `openrouter.json` also counts the HTTP time of retried attempts. Lattice time includes `createAI()` construction and the fetch wrapper's JSON handling, so it is an upper bound. The packed Lattice prompt added 61–83 prompt tokens at the median versus the one-line direct prompt. The paired provider-time difference on that longer prompt had medians from -2398 to 3352 ms. A negative value means the packed call returned sooner. The difference changes sign and is often larger than a short extra prefill can explain, because completion length, including reasoning tokens, moves the provider time by seconds. It is reported separately and is not added to Lattice processing time.

`thinkingmachines/inkling-small:free` is in the public catalog at price 0. Every measured call was rejected with HTTP 403 and the message that the model is only available on agentic harnesses, so this row has no latency sample. `liquid/lfm-2.5-2.6b:free` returned HTTP 504 on the single-agent call (Provider returned error). That call is a provider failure, not a receipt or citation result. `liquid/lfm-2.5-2.6b:free` returned HTTP 504 on the crew call (Provider returned error). That call is a provider failure, not a receipt or citation result.

On a 16×16 solid red PNG, `nvidia/nemotron-3.5-super-vl:free` answered red from a direct `image_url` part: yes (answer red). The Lattice image artifact answered red: no (answer blue). The PNG bytes were present in the Lattice request body: yes. The direct response provider was Nvidia. This vision call set `reasoning.effort=none`. The text arms use reasoning.effort=low. On this vision call, effort low and effort minimal each spent 8192 completion tokens on reasoning and returned empty content. This recorded call sets reasoning.effort=none. Lattice inlines the PNG in the text JSON. The direct arm sends an image_url part. Lattice's OpenAI-compatible request builder, which the openai, xai, openrouter, and lm-studio adapters share, sends each artifact as a text part holding JSON and never emits an image part. A data-URL string is inlined as text and binary bytes are dropped, so the model received no image part, and the Lattice arm answered blue for a red PNG.

TABLE VII

SHORT-PROMPT LATENCY AND THE RAG, OUTPUT-BINDING, TICKET, AND CREW CHECKS. SIGNED MS IS THE MEDIAN OF LATTICE PROCESSING PLUS THE PROVIDER TIME OF THE ATTEMPT THAT RETURNED. LATTICE MS IS THE MEDIAN SIGNED PROCESSING TIME, EXCLUDING PROVIDER TIME, RETRIES, AND BACKOFF. DRIFT CELLS ARE UNIQUE SUCCESSES / SUCCESSES / ATTEMPTS. OUTPUT BINDING COUNTS RECEIPTS THAT VERIFIED AND WHOSE SIGNED OUTPUT HASH MATCHED THE KEPT ANSWER AND CHANGED AFTER A ONE-CHARACTER EDIT, AMONG CALLS THAT RETURNED A RECEIPT. RECEIPT COLUMNS ARE THE SINGLE-AGENT RETURN AND THE CREW RETURN.

Model	Provider	Plain ms	Signed ms	Lattice ms	Drift 0	Drift 0.7	Cited	Output binding	Ticket leak	Agent rcpt	Crew rcpt
llama3.2:3b local	Ollama	278	559	2.45	1/3/3	1/3/3	3/3, 3/3	6/6	no	0	4
`minimax/minimax-m3.1-flash:free`	Minimax	3677	1489	1.69	3/10/10	1/10/10	10/10, 10/10	20/20	no	0	4
`xingchen-agi/xing4.0-29b-a4b:free`	Phala	3124	4812	1.21	4/8/10	3/3/10	8/8, 3/3	11/11	no	0	3
`nvidia/nemotron-3.5-super-vl:free`	Nvidia	675	845	1.20	1/10/10	2/10/10	10/10, 10/10	20/20	no	0	4
`nvidia/nemotron-3.5-lightning:free`	Nvidia	1319	2701	1.10	4/10/10	6/10/10	10/10, 10/10	20/20	no	0	3
`dots-studio/dots-3-note-preview:free`	AtlasCloud	1145	1275	1.50	2/10/10	3/10/10	10/10, 10/10	20/20	no	0	4
`liquid/lfm-2.5-2.6b:free`	Liquid	4179	4649	1.65	3/10/10	2/10/10	10/10, 10/10	20/20	no	fail	fail
`thinkingmachines/inkling-small:free`	—	—	—	—	—	—	—, —	—	—	—	—
`poolside/laguna-s-2.1:free`	Poolside	715	1413	1.56	1/10/10	4/10/10	10/10, 10/10	20/20	no	0	4

TABLE VIII

PURE LATTICE PROCESSING ON THE SIGNED SHORT PROMPT. THE PERCENTAGE IS LATTICE MS OVER LATTICE MS PLUS THE PROVIDER TIME OF THE ATTEMPT THAT RETURNED; RETRIED ATTEMPTS, BACKOFF, AND PACING ARE EXCLUDED. EXTRA TOKENS ARE LATTICE PROMPT TOKENS MINUS THE DIRECT PROMPT TOKENS. PROVIDER DELTA IS THE SIGNED PROVIDER TIME MINUS THE DIRECT PROVIDER TIME, IN MILLISECONDS, AND IS THE LATENCY OF THE LONGER PACKED PROMPT.

Model	ms p50	ms p95	% p50	% p95	Extra tokens	Provider delta
llama3.2:3b local	2.45	14.96	0.439	2.469	—	—
`minimax/minimax-m3.1-flash:free`	1.69	3.66	0.218	0.327	63	-2398
`xingchen-agi/xing4.0-29b-a4b:free`	1.21	3.50	0.032	0.033	83	3352
`nvidia/nemotron-3.5-super-vl:free`	1.20	2.03	0.138	0.260	80	169
`nvidia/nemotron-3.5-lightning:free`	1.10	1.96	0.053	0.082	82	1703
`dots-studio/dots-3-note-preview:free`	1.50	1.90	0.123	0.141	74	59
`liquid/lfm-2.5-2.6b:free`	1.65	4.09	0.035	0.076	61	1005
`thinkingmachines/inkling-small:free`	—	—	—	—	—	—
`poolside/laguna-s-2.1:free`	1.56	4.01	0.142	0.266	76	318

Document-level precision, recall, and F1 use the harness overlap rule: a document is a true positive when any predicted span of that type overlaps a gold span. An undefined F1 means there were no positive predictions, or no gold documents and no false positives. spaCy `en_core_web_sm` emits none of the four labels, so its precision and F1 are undefined and its recall is zero wherever a gold span exists. Figure 7 omits those undefined bars.

The rule is not symmetric across tools. For each type, Lattice contributes every occurrence of the one substring `detect()` returns, while the other tools contribute every span they emit. When Lattice's one match falls on a different number, the document is a false positive and a false negative for Lattice, while a multi-span tool can still score it from another span. GLiNER [31] is the general `urchade/gliner_small-v2.1` model, not a PII fine-tune, and its processor truncated each document to 384 tokens, while LLM Guard [32] scanned 1,500-character windows with 200 characters of overlap. LLM Guard ran its default Anonymize scanner with recognizer `Isotonic/deberta-v3-base_finetuned_ai4privacy_v2`, which was fine-tuned on ai4privacy, so its ai4privacy scores can be optimistic. A single process was OOM-killed on long inputs, so each batch of 250 documents ran in its own process with a fresh Vault. Raw SSN labels are mapped when the NER map returns them unchanged. Presidio [33] used its pattern and spaCy [34] recognizers for its four in-scope entities, with `en_core_web_sm`

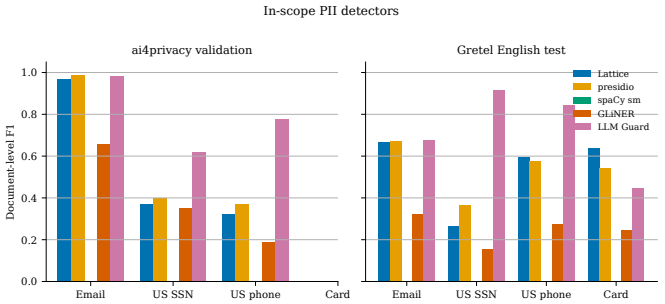


Fig. 7. Document-level F1. A missing bar is an undefined F1.

rather than its default `en_core_web_lg`. Lattice is an in-process JavaScript regex and the other detectors are Python pipelines, so Figure 8 compares runtimes as well as detectors. The Lattice rows were measured with the detectors at commit `a8cc091`; Section VI-C describes how the current detectors differ.

guardrails-detect-pii is from docs, not measured: ImportError: cannot import name 'DetectPII' from 'guardrails.hub' nemo-guardrails is from docs, not measured: nemoguardrails is not installed, and en_core_web_lg is not downloaded. The built-in sensitive-data rails call Presidio, which is scored on its own. The library also documents a separate GLiNER rail.

On ASQA dev [29] (n=948), a case-sensitive Wikipedia title substring matches 525 long answers and a case-insensitive

TABLE IX

ai4privacy English validation [27], document level. Latency is the median milliseconds per document. Lattice rows use the detectors at commit `a8cc091`.

Tool	Email			US SSN			US phone			Card			ms
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	
Lattice	0.955	0.979	0.967	0.325	0.436	0.372	0.316	0.333	0.324	0.000	—	—	0.007
presidio	0.974	0.999	0.986	0.334	0.502	0.401	0.295	0.504	0.372	0.000	—	—	9.413
spaCy sm	—	0.000	—	—	0.000	—	—	0.000	—	—	—	—	8.199
gliner-small	0.695	0.625	0.658	0.283	0.458	0.350	0.228	0.161	0.189	0.000	—	—	52.862
llm-guard-anonymize	0.965	0.999	0.982	0.991	0.450	0.619	0.650	0.962	0.776	0.000	—	—	142.115

TABLE X

Gretel English finance test [28], same document-level rule. Gold credit-card spans in this file use the label `credit_card_number`. Lattice rows use the detectors at commit `a8cc091`.

Tool	Email			US SSN			US phone			Card			ms
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	
Lattice	0.522	0.934	0.670	0.161	0.771	0.266	0.521	0.702	0.598	0.881	0.500	0.638	0.020
presidio	0.526	0.934	0.673	0.244	0.729	0.366	0.458	0.786	0.579	0.833	0.405	0.545	25.627
spaCy sm	—	0.000	—	—	0.000	—	—	0.000	—	—	0.000	—	21.623
gliner-small	0.237	0.511	0.324	0.090	0.557	0.155	0.338	0.234	0.277	0.165	0.500	0.248	156.490
llm-guard-anonymize	0.528	0.952	0.679	0.984	0.857	0.916	0.757	0.952	0.843	0.361	0.581	0.446	426.513

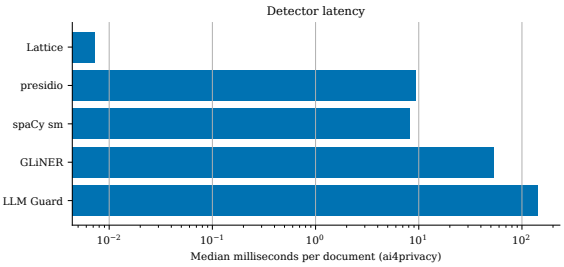


Fig. 8. Median detector time per ai4privacy document. The axis is logarithmic. Lattice runs in-process in JavaScript; the other detectors are Python pipelines.

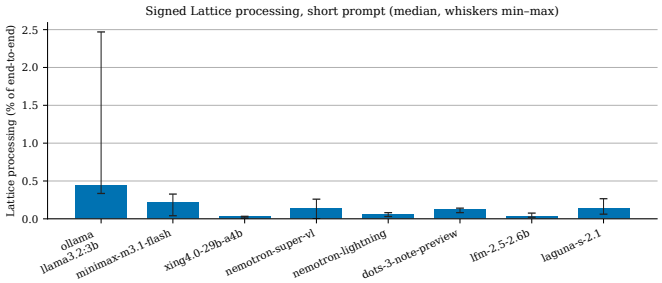


Fig. 9. Signed Lattice processing time as a percentage of end-to-end time, where end-to-end time is Lattice processing plus the provider time of the attempt that returned. Retried attempts, backoff, and pacing are excluded. The bar is the median and the whiskers run from the minimum to the maximum. The local bar is wall time minus the localhost HTTP call in the Ollama run of Section VI-C.

substring matches 605. Square-bracket citations appear in 2 answers and overlap a gold title in 0. `mustCite` accepts a gold title in 7. These three checks are different predicates. The `mustCite` count was measured at commit `a8cc091`, where an inline `[name]` or `(name)` marker containing the title counted as a citation by default. The current default never reads the answer text, so it accepts a gold title in none of these answers unless `inline: true` is passed. A local cross-encoder, `typeform/distilbert-base-uncased-mnli`, scored 696 of 948 pairs and labeled 296 entailed (empty 252, NEUTRAL 211, CONTRADICTION 189, ENTAILMENT 296). Median classification time was 820.0 ms. The premise is the gold knowledge text truncated to 1,800 characters and the hypothesis is the long answer. Pairs with an empty premise or answer were not scored. This is a support label, not a citation-string check.

LiteLLM [3] here is an in-process client, not a proxy process. Its router sent eight short prompts to two identical Ollama deployments and returned 1 distinct route across 8 calls. That is a weak determinism check. The other wrappers are a LangChain callback handler [2], OpenLLMetry's in-process spans [35], and a local Portkey gateway [36]. Helicone and Langfuse were not started. The latency rows are small samples (n=3–5 per arm). Each wrapper ran its calls in its own block, not interleaved with the direct arm, against `llama3.2:3b` on a CPU-only host. The min–max ranges overlap, so the p50 order reflects generation noise, not wrapper overhead.

VII. Discussion

Signed capability receipts change what is possible after a run. An operator can hand a receipt and the kept output to an auditor who verifies the receipt with a public key and checks the output against it offline, without access to the original system. The check shows that the output is the

TABLE XI

LOCAL WRAPPERS AROUND OLLAMA. MARKER DETECTION SCANS THE CAPTURED LOG OR SPAN. PORTKEY'S LOG ROUTE FAILED, SO THAT ROW DOES NOT SHOW WHAT A STORED LOG WOULD CONTAIN. NONE OF THESE LOGS IS SIGNED.

System	Status	n	p50 ms	min–max ms	Bytes	Markers in capture	Offline verify	Signed output binding
ollama-direct	measured	5	1060.2	435–1833	—	—	—	—
litellm	measured	5	3964.8	587–6363	2408	prompt stored	no	no
langchain-callback	measured	5	1936.3	844–8321	6893	yes	no	no
openllmetry	measured	3	1253.1	1211–1329	3346	yes	no	no
portkey-gateway	measured	5	6281.0	1099–8688	1953	log unread	no	no
helicone	from docs, not measured	—	—	—	—	—	—	—
langfuse	from docs, not measured	—	—	—	—	—	—	—

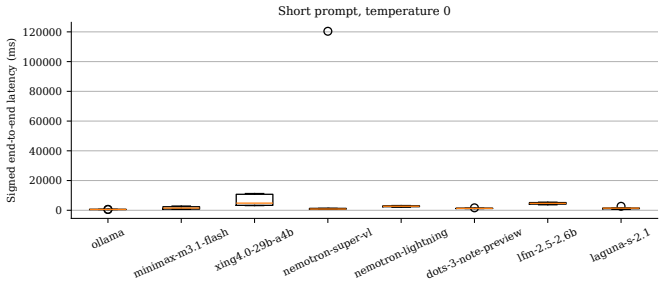


Fig. 10. Signed short-prompt latency. Ollama is five trials. Each hosted row is the successful signed trials, as Lattice processing plus the provider time of the attempt that returned.

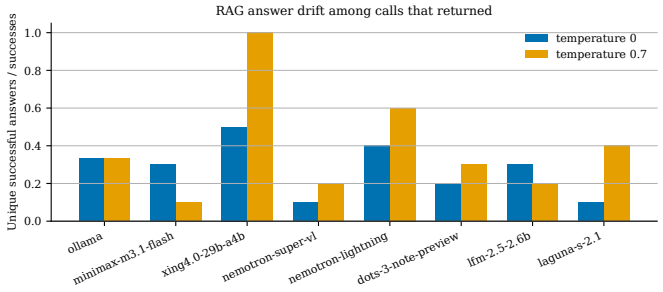


Fig. 11. Distinct successful RAG answers divided by the number of successful calls. Ollama used n=3. Hosted rows use the successes recorded for that model.

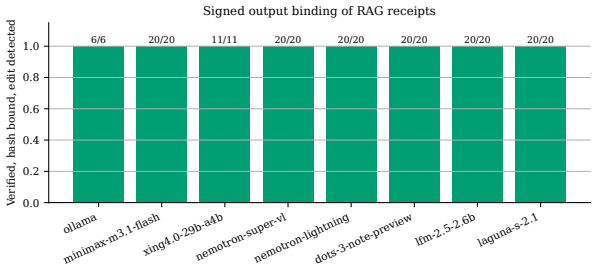


Fig. 12. Signed output binding on receipts from calls that returned. A receipt counts when it verified, its signed output hash matched the kept answer, and a one-character edit of that answer changed the hash. `replayOffline` returns the outputs the caller supplies, so this is a hash-binding check, not a re-run of the model. Rate-limited calls with no receipt are left out of the denominator.

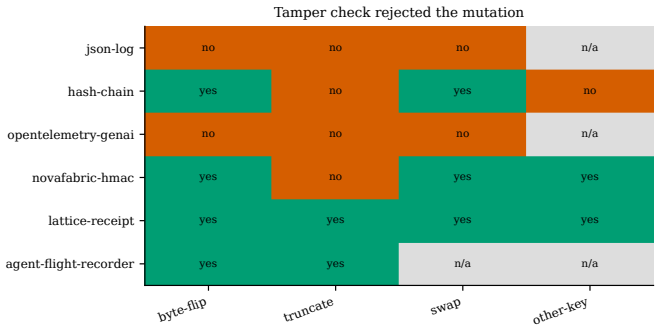


Fig. 13. Whether the system's own check rejects the mutation. Gray means that class was not applied. Truncating a hash chain leaves a valid prefix. A byte flip inside a JSON string, a dropped tail record, and a swapped response all remain valid JSON, so a parser does not catch them.

one the key holder signed, not that the key holder reported the run honestly. A dispute about what a system produced becomes a check against a signed artifact rather than an argument about log files. A continuous integration pipeline can use the `eval` subcommand to gate regressions in cost, in determinism, and in quality across a corpus of receipts, which makes a model or prompt change a reviewable event with measurable consequences. The package itself is published with provenance, so an operator can check that the runtime it deploys was built from the published source. A receipt does not record which build of the runtime produced it.

The design has also been tested from outside. An independent evaluation harness run against release 1.5.1 [37] found three gaps: a violated no-personally-identifiable-information invariant copied the matched substring into the signed evidence while the redaction manifest claimed it had been redacted; receipts carried no freshness or audience binding, so a valid receipt could be replayed to a different relying party at any later time; and crew receipts formed a tree that no verifier checked as a whole. Release 1.6.0 [38] added evidence redaction, the audience, nonce, and expiry fields, and `verifyChain`. Matched personal data is rewritten to a detector name, a length, and a mask before signing. An issuer's `expiresAt` is always enforced, while audience, nonce, maximum age, and the schema-version floor remain verifier opt-ins, so an older receipt still verifies. `verifyChain` checks every link, the single root, and parent reachability, and its failures are a `VerifyChainErrorKind`, not one of the ten `VerifyErrorKind`

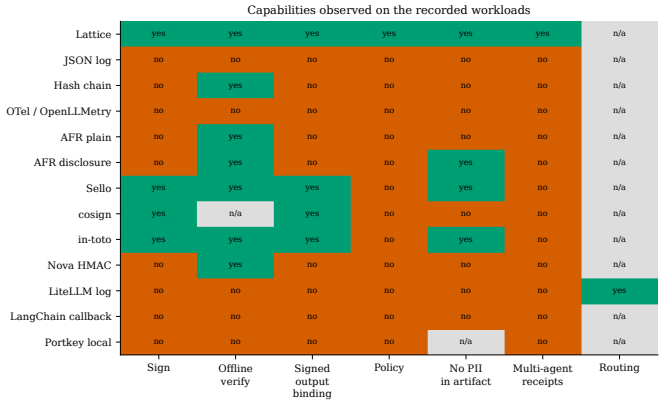


Fig. 14. Capabilities observed on these workloads. Gray is not measured here. Signed output binding is yes when the system signs the record and its own verifier rejected a byte edit of the stored record; for Lattice the signed output hash is checked by re-hashing the kept answer. cosign's offline-verify cell is gray because the measured `verify-blob` checked the Rekor log online. LiteLLM routing is eight calls across two identical local deployments, which returned one route. Lattice routing is gray because each live call used one configured provider. Portkey's stored-log endpoint returned HTTP 500, so the PII cell is gray. LiteLLM's callback log stored the prompt text. The single-agent Lattice return carried no receipt; the crew return did.

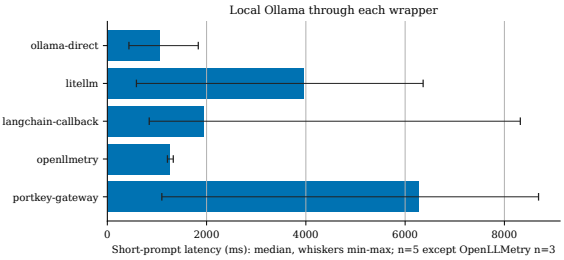


Fig. 15. Short-prompt latency through each local wrapper: median, with whiskers from the minimum to the maximum. OpenLLMetry used n=3. The other measured rows used n=5. The arms ran one after another on a CPU-only host, so the order is not a ranking of wrapper overhead.

values. An open chain fails on tail truncation only when the caller passes an expected count; otherwise the result carries a `tail-truncation-unchecked` warning. A crew run closes its chain with a receipt that lists every other member, so a dropped leaf fails verification without that count, and verifiers in the 1.6.x releases do not enforce that closure (Section V-G). `lattice verify` and `lattice receipt verify-chain` print `WARN` lines for commitments that were not checked, such as an audience the caller did not pass or an open chain verified without a count, and, unless `--quiet` is set, `INFO` lines for optional checks the receipt does not use. The commit and hardware of each harness run are recorded in Appendix XI and in `eval/results/`.

Three design tensions are worth naming. The first is determinism against model nondeterminism. Models are not deterministic, so Lattice does not promise to reproduce a live call. It promises to bind the input-artifact hashes and a kept output to a signed record, and it offers a layered determinism check in the evaluation tool for free form text. The second is redaction against reproducibility. A receipt that redacts a secret cannot reproduce a run that depended on that secret value, so redaction is scoped to material that must never be shared, and the redaction manifest makes the omission explicit and signed. The third is modularity against uniformity. Provider-only consumers can opt into native tools and structured outputs, while the default runtime and agent paths preserve their existing portable behavior. We accept this explicit split because parity is what makes a receipt portable.

VIII. Limitations

Several limitations follow directly from the design. Replay checks a kept output against the signed output hash and does not call a model, so a receipt is evidence about a past run rather than a way to obtain new model behavior. Semantic determinism for free-form text relies on a layered judge in the evaluation tool, which is more expensive and less crisp than an exact hash match. The capability registry is a point-in-time snapshot refreshed by continuous integration, so a brand new model may be absent until the next refresh, although capability negotiation can refine a profile at call time.

A receipt signs less than a full record of the run. It commits to input-artifact hashes, the output hash, the contract hash, and the selected route, but not to the task text, the packed prompt, the policy, or the routing candidates and reasons, so two runs with different instructions but the same artifacts, output, and token usage produce receipts that differ only in identifiers and times. Receipts are signed by the issuing runtime, so they show integrity and the signing key, not that the issuer reported the run honestly. The hashes are unsalted, so a low-entropy input or output can be confirmed from a receipt by guessing. Single-agent runs return no receipt: `ai.runAgent` sends its checkpoint receipts to a tracer only, and only `ai.run` and crews return receipts to the caller. The five OpenAI-shaped adapters send artifacts as JSON text, drop values that are not strings, such as binary image bytes and JSON objects, and still hash the dropped input, so for those adapters an input hash does not show that the model received the input; in the hosted vision probe the Lattice arm answered blue for a red image while a direct image part answered red (Section VI-D). When a configured signer fails, the runtime does not hide the gap: by default the result carries a receipt error field and an event, and an operator can instead configure the run to fail.

The default personally identifiable information detectors are regular expressions over four United States and email formats documented in the source. Names, addresses, and non-United States formats are not detected. Identifier labels and type cues trade recall against precision: a real number right after a label such as `ref`, or a bare number without its cue, is missed, and long Luhn-valid identifiers are flagged. On public data the social security and phone detectors confuse identifiers of the same shape, such as passport numbers, and score far below the synthetic harness (Table IX). Those public-data rows were measured with the detectors at commit `a8cc091`, which differ from the rules above as Section VI-C describes. The must-cite invariant is syntactic. By default it reads only citation lists,

and its prose options check markers, not whether the cited artifact supports the claim. Long-form ASQA answers carry no citation list, so the default accepts a gold title in none of them; the count in Section VI-C was measured at commit `a8cc091`, where an inline marker containing the name counted by default.

No replay cache ships with the verifier, so a relying party that must accept a receipt only once keeps its own record of accepted content identifiers. An open chain warns on tail truncation unless an expected count is passed. A crew's closing receipt makes a dropped leaf a count mismatch without an out-of-band count, but a verifier from the 1.6.x releases does not enforce it, and a missing closer leaves the closed chain unverifiable. Finally, external audit helpers can sign and replay evidence produced elsewhere, but they cannot prove that the external executor used the claimed provider unless that executor supplies trustworthy raw envelopes. The hosted measurements use free endpoints. They rate-limit, they are not a paid-latency sample, and a model that rejects ordinary chat calls is reported as unavailable rather than given a latency. Paired provider times are too noisy to treat as the cost of the extra prompt tokens Lattice packs.

IX. Future Work

Several extensions build naturally on the receipt primitive. Receipts can commit to a hash of the task, the packed prompt, and the policy, and can use salted or keyed commitments for low-entropy values. The OpenAI-shaped adapters can send native image parts, or fail a run whose artifact they cannot package, so that an input hash always names an input the provider received. `ai.runAgent` can return its checkpoint receipts and a closing receipt to the caller. External audit can become stronger when executors attach provider-side attestations or gateway signed envelopes. The run event vocabulary can be exported more broadly through OpenTelemetry [39] so receipts and traces share a vocabulary across organizations. Key management can be hardened with key management service backed signers and rotation policies. Realtime audio and video sessions need a distinct session surface and a session summary receipt, since a bidirectional stream differs structurally from a request and response. Finally, the modular subpaths can be validated against more downstream runtimes so adoption does not require replacing an existing orchestration stack.

X. Conclusion

Production systems built on large language models need runs that an auditor can verify and an operator can reproduce. We presented Lattice, a capability-first runtime in which a signed model run emits a capability receipt that is canonicalized, redacted before signing, and signed inside a Dead Simple Signing Envelope, backed by a deterministic and inspectable router and by output invariants with terminal semantics. The receipt binds the input-artifact hashes, the output hash, the contract hash, and the selected route; it does not record the task text, the policy, or why the route was chosen. A verifier can check a receipt with a public key and confirm offline that a kept output hashes to the signed value. The implementation ships publicly with supply-chain provenance, a seven-provider parity contract over one receipt format, a registry of 332 model profiles, modular adoption paths, a measured signing and verification cost (Table V), and a test suite that encodes the guarantees as invariants. By making a signed receipt a byproduct of every `ai.run` call and crew run that has a signer, Lattice turns auditability from after-the-fact instrumentation into a property of the runtime itself.

XI. Reproducibility notes

The benchmark in `paper/bench/results.json` was produced by `scripts/bench-receipts.mjs` on Node v24.22.1 (linux 6.12.94+ x64), processor Intel(R) Xeon(R) Processor, Lattice 1.6.0, commit `4549334`. Provider: mocked (createFakeProvider), no network.

The packages `@full-self-browsing/lattice` and `@full-self-browsing/lattice-cli` are published on the public npm registry with an OpenID Connect trusted publisher and with supply-chain provenance attestations in the spirit of SLSA [9]. A tagged GitHub release accompanies each publish. The runtime targets Node 24 or newer. Modular subpaths for audit, context, artifacts, routing, tools, eval, and core are also exercised as Node-20-style consumers in the test suite.

The command-line tool returns exit status 0 for success, 1 for a verification or test failure, and 2 for a load or configuration failure. A successful `verify` or `receipt verify-chain` prints advisory lines on standard error, each with an advisory code: `WARN` lines always and `INFO` lines unless `--quiet` is set. The process still exits 0, so a caller gates on the codes. `receipt verify-chain` also accepts `--require-closure` and `--partial`, and a malformed closing receipt fails with a typed `FAIL kind=` line.

The offline evaluation harness lives in `eval/`. `eval/run-all.sh` rebuilds the packages and writes `eval/results/`, including hardware and the git commit. Provider calls in that harness are in-process fakes. No API key is required.

Acknowledgment

The evaluation uses the ai4privacy pii-masking-300k dataset [27] under its terms for academic, non-commercial use with acknowledgement, the Gretel synthetic PII finance dataset [28], and the ASQA dataset [29]. The committed results hold aggregate scores, not dataset records.

References

[1] Vercel, "AI SDK," Software, online documentation, 2024, https://sdk.vercel.ai.

[2] LangChain Inc., "LangChain and LangGraph," Software, online documentation, 2023, https://www.langchain.com.

[3] BerriAI, "LiteLLM: Call 100+ LLM APIs in the OpenAI format," Software, online documentation, 2023, https://github.com/BerriAI/litellm.

[4] OpenAI, "OpenAI agents SDK," Software, online documentation, 2024, https://github.com/openai/openai-agents-python.

[5] Anthropic, "Model context protocol (MCP)," Online specification, 2024, https://modelcontextprotocol.io.

[6] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *Proceedings of the 28th USENIX Security Symposium (USENIX Security)*. USENIX Association, 2019.

[7] Secure Systems Lab and the in-toto project, "Dead simple signing envelope (DSSE) specification," Online specification, v1.0, 2021, https://github.com/secure-systems-lab/dsse.

[8] Z. Newman, J. S. Meyers, and S. Torres-Arias, "Sigstore: Software signing for everybody," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2022.

[9] Open Source Security Foundation (OpenSSF), "Supply-chain levels for software artifacts (SLSA) framework," Online specification, 2023, https://slsa.dev.

[10] J. Samuel, N. Mathewson, J. Cappos, and R. Dingledine, "Survivable key compromise in software update systems," in *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS)*. ACM, 2010.

[11] A. Rundgren, B. Jordan, and S. Erdtman, "JSON canonicalization scheme (JCS)," RFC 8785, Internet Engineering Task Force (IETF), 2020, https://www.rfc-editor.org/rfc/rfc8785.

[12] T. Bray, "The I-JSON message format," RFC 7493, Internet Engineering Task Force (IETF), 2015, https://www.rfc-editor.org/rfc/rfc7493.

[13] S. Josefsson and I. Liusvaara, "Edwards-curve digital signature algorithm (EdDSA)," RFC 8032, Internet Engineering Task Force (IETF), 2017, https://www.rfc-editor.org/rfc/rfc8032.

[14] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, "Model cards for model reporting," in *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*. ACM, 2019.

[15] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. D. III, and K. Crawford, "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.

[16] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.

[17] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology — CRYPTO '87*. Springer, 1988.

[18] J. Figuera, "Notarized agents: Receiver-attested confidential receipts for AI agent actions," arXiv:2606.04193 [cs.CR], Jun. 2026, https://arxiv.org/abs/2606.04193.

[19] L. Bindschaedler, Q. Botha, and C. Siebenbrunner, "Agent flight recorder: Tamper-evident audit trails with on-chain anchoring for long-horizon tool-using agents," in *Proc. IEEE Int. Conf. Blockchain Computing and Applications (BCCA)*, 2026, arXiv:2609.01931, https://arxiv.org/abs/2609.01931.

[20] M. Seyedkazemi Ardebili, "NovaFabric: Tamper-evident, replayable evidence for autonomous AI agent runs," arXiv:2609.12582 [cs.CR], Sep. 2026, https://arxiv.org/abs/2609.12582.

[21] M. Arslan, "Janus: Evidence-before-effect sagas and offline-verifiable provenance for agentic LLMs," arXiv:2609.38266 [cs.CR], Sep. 2026, https://arxiv.org/abs/2609.38266.

[22] M. Li, "From cloud-native to trust-native: A protocol for verifiable multi-agent systems," arXiv:2507.22077, Jul. 2025, vision paper. https://arxiv.org/abs/2507.22077.

[23] B. Laurie, A. Langley, and E. Kasper, "Certificate transparency," Internet Engineering Task Force, RFC 6962, Jun. 2013.

[24] B. Laurie, E. Messeri, and R. Stradling, "Certificate transparency version 2.0," Internet Engineering Task Force, RFC 9162, Dec. 2021.

[25] L. Turlapati, "Lattice: Capability runtime SDK for multimodal AI applications (source code)," GitHub repository, 2026, https://github.com/fullselfbrowsing/Lattice.

[26] The Standard Schema project, "Standard schema specification," Online specification, 2024, https://standardschema.dev.

[27] Ai4Privacy, "pii-masking-300k," Hugging Face dataset, revision c8c77895a005822682b66ab547fc0422579bc1d3, 2024, academic research and non-commercial use with acknowledgement; no redistribution. https://huggingface.co/datasets/ai4privacy/pii-masking-300k.

[28] Gretel.ai, "Synthetic PII finance multilingual (gretelai/synthetic_pii_finance_multilingual)," Hugging Face dataset, revision 7b844d16738527a04264f50214cb426a4cea0897, 2024, Apache-2.0. https://huggingface.co/datasets/gretelai/synthetic_pii_finance_multilingual.

[29] I. Stelmakh, Y. Luan, B. Dhingra, and M.-W. Chang, "ASQA: Factoid questions meet long-form answers," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2022, pp. 8273–8288, data: din0s/asqa, revision 084060f16b46f3165318f760b2339208b19a0bde, Apache-2.0.

[30] The Sigstore Authors, "cosign: Code signing and transparency for containers and binaries," Software, Apache-2.0 license, 2021, https://github.com/sigstore/cosign.

[31] U. Zaratiana, N. Tomeh, P. Holat, and T. Charnois, "GLiNER: Generalist model for named entity recognition using bidirectional transformer," in *Proceedings of NAACL 2024 (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024, pp. 5364–5376.

[32] Protect AI, "LLM guard: The security toolkit for LLM interactions," Software, MIT license, 2023, https://github.com/protectai/llm-guard.

[33] Microsoft, "Presidio: Data protection and de-identification SDK," Software, MIT license, 2018, https://github.com/microsoft/presidio.

[34] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-strength natural language processing in Python," Zenodo, 2020, https://spacy.io.

[35] Traceloop, "OpenLLMetry: Open-source observability for LLM applications, based on OpenTelemetry," Software, Apache-2.0 license, 2023, https://github.com/traceloop/openllmetry.

[36] Portkey AI, "Portkey AI gateway," Software, MIT license, 2024, https://github.com/Portkey-AI/gateway.

[37] Trace, "Eval harness findings: matched PII signed into no-PII receipts, receipt replay and chain gaps, and 8 more issues," GitHub issue #21, fullselfbrowsing/Lattice, 2026, opened by GitHub user Trace824. https://github.com/fullselfbrowsing/Lattice/issues/21.

[38] L. Turlapati, "Receipt hardening for issue #21," GitHub pull request #22, fullselfbrowsing/Lattice, released in version 1.6.0, 2026, https://github.com/fullselfbrowsing/Lattice/pull/22.

[39] Cloud Native Computing Foundation, "OpenTelemetry specification," Online specification, 2023, https://opentelemetry.io/docs/specs.